CMSI3300.02 - Homework 4

Objective
Overview
    Machine Learning Pipeline
Solution Skeleton
Task 1: Toxicity Filter
Task 2: Guessing Salaries
Tests & Grading
Submission
    What
    How

Objective

This assignment demos some machine learning classics, and explores some applied approaches to our supervised learning problems!

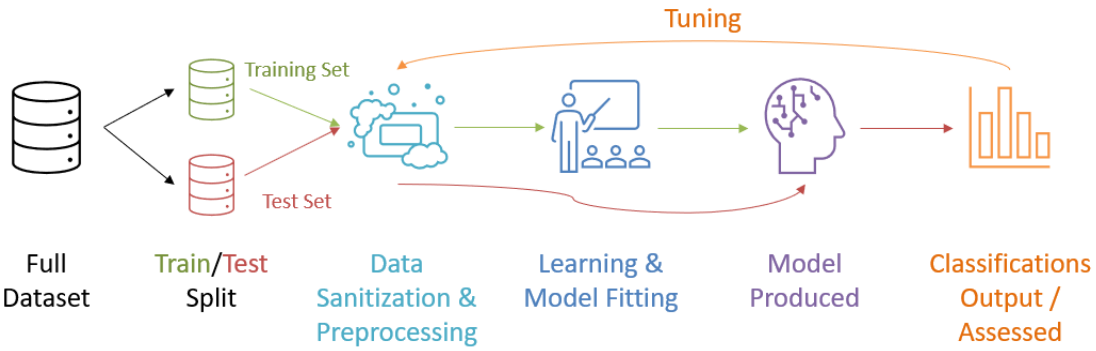
In this assignment, you'll:

- Work with real-world (and therefore: messy!) data in two classification tasks.
- Use some of the most popular Python libraries for Machine Learning (Pandas, Sklearn)
- Address some real-world issues in data sciences, like sanitizing data and dealing with imbalanced classes.

Overview

Machine Learning Pipeline

Here's what the process will look like from a bird's-eye view:



Solution Skeleton

Start with the solution skeleton in-hand!

🔗 GitHub Classroom Link

Contents Included in the skeleton are:

- `toxicity_filter.py`, which will contain the logic for designing a Naive Bayes Classifier to detect toxic comments in online forums, created from a ton of Wikipedia Talk page examples that have been hand-labeled as toxic / not.
- `salary_predictor.py`, which will contain the logic for designing a Logistic Regression Classifier to predict the salary of individuals based on certain reported characteristics.
- `classifier_tests.py`, containing some basic unit tests to guide you through each of the processes above.
- `/dat/` is a folder containing csv data files with the features and labels for each of the two classifier models we'll be building.

!! One of our datasets `wiki_talk.csv` includes ***actual*** user comments from Wikipedia Talk pages and thus, showcases some of the worst dregs of society that humanity has to offer. Comments may have vulgar and offensive language, which is what we'll be trying to detect and prevent, so open this file and read *at your own risk!* No part of this assignment will ***require*** you to read through them, but you if you don't have a weak stomach, you might want to take a look.

Setup

Use Python 3.X for this assignment (for X greater than or equal to 9).

Before getting started, you'll also need the following Python packages, which can be installed via `pip` / `pip3`:

- `pandas`: used to manage the DataFrames, which we remember from a previous assignment as the way we can manipulate the data columns / rows and feed into our learning algorithms.
- `sklearn`: a popular library replete with a bunch of different ML models, including all of the ones we learned about in class!

  To install this dependency, `pip install scikit-learn`

Task 1: Toxicity Filter

This assignment will walk you through the creation of a simple Naive Bayes Classifier for filtering toxic messages from a web forum, as trained on comments from the Wikipedia Talk pages!

The following will walk you through the process of training your basic NBC! Follow along in your `toxicity_filter.py` while you consult the documentation at each step.

✅ Problem 1: Know Your Data

💡 **Tools used in this section:**

🔗 Pandas `read_csv`

🔗 Sklearn `train_test_split`

However, if you want to dive in, note the following in `/dat/wiki_talk.csv`

- Note the column names: `comments` for our inputs, X and `toxic` as our classes: 0 = non-toxic, 1 = toxic.
- Note how many rows: that's a LOT of data!
- [At your own risk] Skim the pairings of messages to their labels: do you agree with most of the human labeling here?

Example

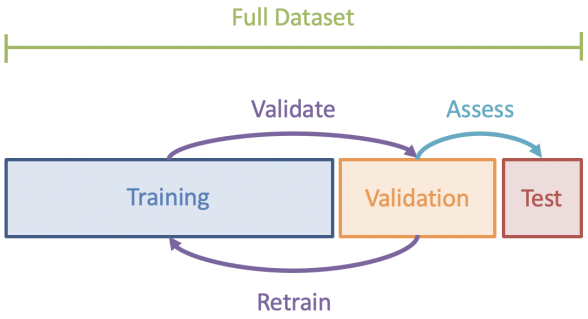
For the less brave, here are a couple of hand-selected examples of each class:

Comment (Input X)	Toxicity (Class y)
There's no need to apologize. A Wikipedia article is made for reconciling knowledge about a subject from different sources, and you've done history studies and not archaeology studies, I guess. I could scan the page, e-mail it to you, and then you could ask someone to translate the page.	0 (non-toxic)
GET F***** UP. GET F***** UP. GOT A DRINK THAT YOU CANT PUT DOWN???/ GET F*** UP GET F***** UP. I'M F***** UP RIGHT NOW!(* obviously filtered for this audience of gentle readers. Also, always Wiki sober.)	1 (toxic)
All of my edits are good. ***** like you who revert good edits because you're too stupid to understand how to write well , and then revert other edits just because you've decided to bear a playground grudge, are the problem. Maybe one day you'll realise the damage you did to a noble project.(don't ever go full-neckbeard)	1 (toxic)

Next, we'll need to import the data in an easily manipulable format. Pandas DataFrames to the rescue!

The good news is that you already dealt with loading data like this in a previous assignment, so I've given you the loaded data in the `classifier_tests.py`

```
# classifier_tests.py
TOXICITY_DATA = pd.read_csv("../dat/wiki_talk.csv")
```



Now, recall from lecture, and the graphic above, that we need to **split our dataset** into a training and test partition so that we can accurately judge how whatever model we've trained generalizes to samples it's never seen during training.

Note that in class we looked at the data split into a 3rd piece (the validation set): this is something that is usually carved out of the training data in order to iteratively find the best

parameters / hyperparameters and is handled internally to the `sklearn` methods. The "test" data above is the "held-out" data that you will never see during training.

This has also been done for you (in order to make sure things stay consistent) using a nice helper method from `sklearn`:

```
# classifier_tests.py
TOX_TEXT_TRAIN, TOX_TEXT_TEST, TOX_LABELS_TRAIN, TOX_LABELS_TEST = \
train_test_split(TOXICITY_DATA["comment"], TOXICITY_DATA["toxic"], t
est_size=0.3, random_state=1337)
```

Note the arguments specified above:

- `TOXICITY_DATA["comment"], TOXICITY_DATA["toxic"]` the columns in the data frame representing the inputs and outputs, respectively.
- `test_size=0.3` means that 30% of the data will be reserved for testing, and the remaining 70% for training.
- `random_state=1337` is a random-number-generator seed that could have been set to any number, but whatever number it's set to, it'll consistently choose the same pseudo-random set of rows from the data frame to split into training and test, which will make for consistent results while developing.

Now that the data's been loaded and split, we're ready to start making our filter!

✅ Problem 2: Feature Extraction

Implement the following in your ToxicityFilter's `__init__` method!

Here's what we're promising to implement in the constructor -- note the parameters.

```
# toxicity_filter.py
def __init__(self, text_train: pd.DataFrame, labels_train: pd.DataFr
ame):
"""
Creates a new forum comment ToxicityFilter trained on the given
wikitalk messages and their associated toxic/not labels. Performs
any necessary preprocessing before training the ToxicityFilter's
Naive Bayes Classifier. As part of this process, trains and stores
the CountVectorizer used in the feature extraction process.
```

```
Parameters:
    text_train (pd.DataFrame):
        The data frame containing the text of comments on which this
        NBC is to be trained.
    labels_train (pd.DataFrame):
        The labels of 0=non-toxic and 1=toxic corresponding to each
        row (by index) in the text_train dataframe's comments.
"""
```

💡 **Tools you'll need for this section:**

🔗 Sklearn `CountVectorizer`

We're almost ready to train! *cue Rocky soundtrack*... but must do a bit of feature extraction to get things ready for our NBC.

So the DataFrame's in order, but our preprocessing work isn't done: the text of each comment needs to be converted into a format our NBC will understand.

In particular, since we're training an NBC, which uses the "Bag of Words" parameterization, we want to make sure we give it the words that are most likely to be related to one class over the other.

- Words like "am", "is", "the", etc. are called **stopwords**, since they're common to pretty much every class imaginable, and would pollute our model with many extraneous parameterizations.
- We want to count the number of times the ***meaningful*** words appear such that the ones that appear more with a certain class are weighted higher in the parameters.

As such, examine the API for the `sklearn CountVectorizer` to first create a CountVectorizer, then `fit` it and `transform` our `data["text"]` column such that our text data contains no more of the English stopwords, has some normalization applied, and is now converted into a set of vectorized counts over words.

Important: save this vectorizer as an attribute called `self.vectorizer` -- this is necessary for the validation tests as well as re-using the vectorizer later!

The `fit` and `transform` operations are common in many `sklearn` applications; to understand what's happening:

- `fit` is used to train some model on a dataset; in the case of the CountVectorizer, this means to learn a **vocabulary** of the underlying words in some training set alongside the number of times the words in that vocabulary appear.
- `transform` is used to turn raw input into the featurized input to the classifier. In the case above, this turns each word in the vocabulary into a vectorized representation mapped to the counts.
- `fit_transform` combines the two steps above.

Generally, you will use `fit` and `transform` separately if you want to save the transformer model as, say, an attribute after fitting it, otherwise use `fit_transform` to do both at once.

If your vectorizer's been configured properly, at this point, you should pass the first unit test, which will also print out a few sample words in its learned vocabulary -- try it out! `pytest -k test_toxicity_filter_vectorizer`

One last step here:

Make a variable called `features` that's the result of calling your CountVectorizer's `transform` method on the `text_train` parameter (not as an attribute, you'll just use these to train the NBC).

There are, of course, many more sophisticated preproecessing techniques we could do here, like in making a feature extractor to produce additional features on which to train the model, but we'll stick with just the above for our purposes in this assignment.

✅ **Problem 3: Training the Model**

Implement the following in your ToxicityFilter's `__init__` method!

💡 **Tools you'll need for this section:**

🔗 Sklearn `MultinomialNB`

We'll now use the scikit Naive Bayes Classifier module to train our model on the training set! To do so:

1. Create a new MultinomialNB object (there are some tuning parameters you can add to it later if you'd like, see documentation)

2. Call that object's `fit(training_features, training_classes)` method to learn the NBC parameters (hint: the constructor's parameters)!

Save your learned NBC model as an attribute of your ToxicityFilter -- you'll need it later!

✅ Problem 4: Classification

Implement the following in your ToxicityFilter's `classify` method!

```
# toxicity_filter.py
def classify (self, text_test: list[str]) -> list[int]:
"""
Takes as input a list of raw forum comments, uses the filter's
vectorizer to transform these into the known bag of words, and
then returns a list of classifications, one for each input text.

[!] Note: Should only use the vectorizer's transform method, you
should NOT be using fit_transform which will re-fit it to test data

Parameters:
    text_test (list[str]):
        A list of forum comments comprising the messages that the
        toxicity filter must classify as toxic (y=1) or not (y=0)

Returns:
    list[int]:
        A list of classifications, one for each input message, where
        the index of the output class corresponds to the index of
        input message. The ints represent the classes such that
        y=0=non-toxic and y=1=toxic
      """
```

💡 **Tools you'll need for this section:**

🔗 Sklearn `MultinomialNB`

This one's simple!

With the now-trained feature-extractor and NBC that we constructed in the __init__ method, let's see how to use them.

In your `classify` method, you'll simply:

- Use your vectorizer (remember: saved as an attribute) to transform the input text messages into their vectorized format (which your NBC knows how to deal with).
- Use the NBC's `predict` method to retrieve the labels for each input text message.
- Return the predicted labels as a list.

If implemented properly, we can now test out a couple of examples that *we've* made by hand to make sure its outputs look good! Head on over to the tests and run `pytest -k test_toxicity_filter_classification`

If all looks good there, let's see how we did on the larger test set!

✅ Problem 5: Testing the Model

Examine the ToxicityFilter's `test_model` method! **[This method has been given, you don't need to fill anything out here, but should understand what it's doing]**

```
# toxicity_filter.py
def test_model (self, text_test: pd.DataFrame, labels_test: pd.DataF
rame) -> tuple[str, dict]:
"""
Takes the test-set as input (2 DataFrames consisting of test inputs
and their associated labels), classifies each, and then prints
the classification_report on the expected vs. given labels.

Parameters:
    text_test [pd.DataFrame]:
        Pandas DataFrame consisting of the test rows of comments
    labels_test [pd.DataFrame]:
        Pandas DataFrame consisting of the test rows of labels
        pertaining to each text message

Returns:
    tuple[str, dict]:
```

```
        Returns the classification report in two formats as a tuple:
        [0] = The classification report as a prettified string table
        [1] = The classification report in dictionary format
        In either format, contains information on the accuracy of
        the classifier on the test data.
"""
      prediction = self.classify(text_test.values.tolist())
      return (classification_report(labels_test,prediction, output_d
ict = False),
              classification_report(labels_test,prediction, output_d
ict = True))
```

💡 **Tools used in this section:**

🔗 https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Time to see how we did on the test set! To do so we'll:

1. Call your ToxicityFilter's `classify` method on the `text_test` comments to see how it classifies messages it's never seen before.
2. Print some analytics on how the result of our predictions played out compared to the input `y_test` by printing the results of the `classification_report(y_expected, y_actual)` (where y_expected are the correct answers we input as parameters, and y_actual are your model's predictions from the step above)

To test its efficacy, let's run the classifier across all of our test samples in the following unit test, seeing the classification report as output! `pytest -k test_toxicity_filter_performance`

What we'll see from the classification report are some common, but important, metrics of success (quoting the documentation):

- The **precision** is the ratio tp / (tp + fp) where tp is the number of true positives and fp the number of false positives. The precision is intuitively the ability of the classifier not to label as positive a sample that is negative.
- The **recall** is the ratio tp / (tp + fn) where tp is the number of true positives and fn the number of false negatives. The recall is intuitively the ability of the classifier to find all the positive samples.

Notice that the only difference between precision and recall is a term in the denominator where precision penalizes for more false positives (we said it was toxic when it wasn't) and recall penalizes for false negatives (we said it wasn't toxic when it was).

These are often important metrics where the classes are imbalanced, because, e.g., we could say that in a dataset where $P(Y=0)$=0.99P(Y=0)=0.99 to simply label EVERY data point as Y=1Y=1 and our *accuracy* would be close to 99%, but the precision would thus be lower due to the ratio of false positives.

- The **F-beta score** can be interpreted as a weighted harmonic mean of the precision and recall, where an F-beta score reaches its best value at 1 and worst score at 0. The F-beta score weights recall more than precision by a factor of beta. beta == 1.0 means recall and precision are equally important.
- The support is the number of occurrences of each class in y_expected. This is important to consider for unbalanced datasets (see next section).
- The above are given for each of the individual class labels, 0 = non-toxic and 1 = toxic, as well as through a weighted average based on how many samples there were of each class.

Notice that the precision and recall on class 1 = Toxic is poorer than on class 0 = non-toxic. Why do you think this is? [Hint: look at the number of instances (i.e., support) of each class] and remember the $P(y)$P(y) factor in our NBC decision rule.

If all above went well, this model probably does pretty darn well! The power, albeit simplicity, of the NBC bag-of-words approach!

✅ **Problem 6: Tuning the Model / Training**

[Optional] Play with the following in your ToxicityFilter's `__init__` method!

💡 **Tools you'll need for this section:**

🔗 How to Handle Imbalanced Classes in Machine Learning

Note that in our training data, we had many more instances of non-toxic comments compared to toxic ones; this can be a problem, since we know our NBC weights by $P(Y)$,

and can sometimes mean that the model is not adequately distinguishing between classes so much as it is just guessing the dominant one (which will mean it's still right most of the time).

Read the article linked above for this section, which you're not required to do for this assignment (and will actually tank performance even if it means making a model more capable of distinguishing toxic from non-toxic messages), but is academically interesting, and even shows you how to use scikit's logistic_regression module, which we'll use in the next task!

Now... let's see if you can take the above demo and apply it to another model... or did you overfit to the example above 0_o?

Task 2: Guessing Salaries

In this **classic** supervised learning task, your job will be to use a dataset on demographic and employment information from several decades ago to see if you can predict whether or not someone made more or less than $50k in salary -- capitalism ho!

Note: This is merely an exercise that has you deal with real world, messy data in pursuit of a classification task -- you should reflect on whether or not using data like so to predict something sensitive like salary is ethical given its many possible contexts!

The dataset (`/dat/salary.csv`) on which you will be training your classifiers is a classic in the machine learning literature: some census information relating the features of thousands of adults to a discretization of their income: a single, binary class variable of $Y \in \{\leq 50k, > 50k\} = \{0,1\}$.

The features are then decomposed as follows:

Feature	Data Type	Description / Values
age	Continuous	Age in years (all > 16)
work_class	Categorical	Code pertaining to category / nature of work: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
education	Categorical	Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
education_years	Continuous	Number of years in education.
marital	Categorical	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse

occupation_code	Categorical	Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
relationship	Categorical	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried
race	Categorical	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex	Categorical	Female, Male (this dataset is not modern, so gender was still only collected as binary)
capital_gain	Continuous	Amount gained from the sale of property or investment.
capital_loss	Continuous	Amount lost from the sale of property or investment.
hours_per_week	Continuous	Hours per week worked in the person's occupation.
country	Categorical	Tons of different countries of origin.

Your Task

Your task: intelligently sanitize and preprocess the training data to then train a `logistic_regression` classifier to perform salary prediction!

That said, it's not all a copy-paste walk down easy street compared to Task 1; the training wheels are off, and you have some new challenges to address:

- The data is not perfect nor complete; some data points have missing fields, and you'll need to learn how to deal with them.
- The data is messy -- note that many columns have values with leading / training spaces that must be sanitized!
- The documentation on the relevant scikit modules is good, but requires several passes to understand, as well as understanding the expected parameters and opportunities to improve performance via tuning.
- Apropos, leaving some time to tune how you preprocess and choose the hyperparameters (i.e., the parameters of the training itself) will let you get the most out of your model's performance.

Preprocessing Data

Training / Test data is sometimes not in the format that we need / want it to be in for processing. So, we will often first pre-process it to be in a format that is conducive to training or the particular model that we are using.

It is sometimes said that preprocessing is where 90% of the time is spent on machine learning -- once the data's in a good format, the rest is easy!

Some things to consider:

- **[Messy Data]** notice that many columns' fields have a leading space to them -- obnoxious, and can possibly lead to bugs!
- **[Missing Data]** some fields have missing values, as indicated by `?` in a given data point. What you do with these records is up to you! (see suggestions that follow).
- **[Continuous vs. Categorical Data]** you may wish to change some continuous variables into discrete ones through a transformation; your model will lose expressiveness in this way, but will gain power on the training set -- you'll have to toy with this and investigate the documentation to figure out what's best!

  Hint: this is a Logistic Regression classifier, so it needs every feature extracted to be a number where a number being higher means something tangible!

  Want to take an input column that isn't necessarily ordinal (like the occupation codes) but convert them into something numerical? Consider making each value a separate binary feature that's either 0 or 1 -- Hint: a preprocessing method in the link below will be helpful here!
- **[Feature Selection / Engineering]** some features may not be particularly useful in determining the class; you may want to investigate this ahead of model formation.

Let's take a look at some examples of the above!

✅ Problem 7: Implementing Preprocessing

Head on over to your `salary_predictor.py` and the `preprocess` method for this part!

```
# salary_predictor.py
def preprocess (self, features: pd.DataFrame, training: bool = Fals
e) -> npt.NDArray:
"""
Takes in the raw rows of individuals' characteristics to be used for
salary classification and converts them into numerical features that
can be used both during training and classification by the LR model.

Parameters:
    features [pd.DataFrame]:
        The data frame containing all inputs to be preprocessed
        where the rows are 1 per person to classify and the columns
```

```
        are their attributes that may require preprocessing, e.g.,
        one-hot encoding the categorical attributes like education.

    training [bool]:
        Whether or not this preprocessing call is happening during
        training (i.e., in the SalaryPredictor's constructor) or
        during testing (i.e., in the SalaryPredictor's classify
        method). If set to True, all preprocessing attributes like
        imputers and OneHotEncoders must be fit before transforming
        any features to numerical representations. If set to False,
        should NOT fit any preprocessors, and only use their
        transform methods.

Returns:
    np.ndarray:
        Numpy Array composed of numerical features converted from
        the raw inputs.
"""
```

Reminder: the end goal here: logistic regression requires features to be only vectors of numerical input!

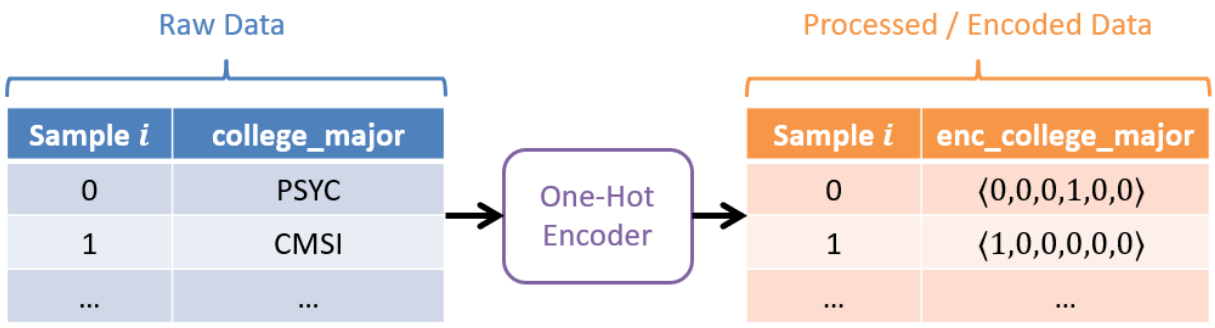
As such, you'll need to perform all sanitization, preprocessing, etc. in the preprocess method.

To aid you in these endeavors, see the scikit library's preprocessing methods (6.3.4 and onwards are most applicable here, remembering that we are using Logistic Regression as our model!):

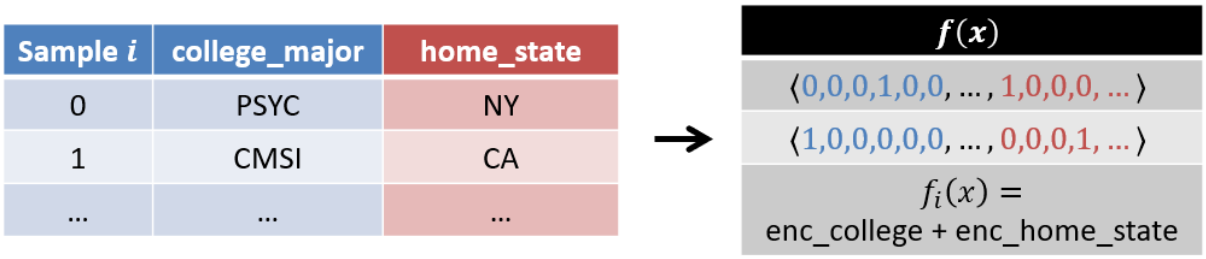
> 💡 Read through the following resources thoroughly, they'll save you a lot of heartache if you (shocker) actually start by reading the documentation!
>
> 🔗 Sklearn Preprocessing Data
>
> 🔗 Logistic Regression for Binary Classification Task
>
> 🔗 A few tutorials

To wit, certain inputs may be **categorical** in nature, like your HomeState or CollegeMajor.

One-Hot Encoders are tools used to convert categorical data into a one-hot vectorized format:



Importantly: whenever you use a One-Hot Encoder on the training data, that SAME encoder must be used to process the test data / other data on which the model is deployed!



In words, from the above:

- Suppose we one-hot encode 2 ordinal variables, one's college major and their home state. Our feature vector is thus a *single* vector composed of those two concatenated.
- The order of the individually encoded variables in the feature vector doesn't matter *as long as that order remains consistent through training and testing*.

Important Reminder: Just like in the Toxicity Filter, all of your preprocessing objects should be trained / fit during TRAINING, but NOT during testing!

Take, for example, the SimpleImputer, whose job it is to fill in any missing data points. How it fills those in is usually a consequence of trends in the training data, e.g., using the "mean" method, it will fill in missing data points for a variable / column using the one that appears with the highest average. As such, you would want to fit the imputer to the average value that would be seen across all training examples (i.e., have it find the average value amongst the training samples), but you would not want to fit it to test data that will be less complete.

In either case (training OR testing), however, its transform method will take in some data with missing variable values, and complete them using the averages it learned during

fitting, so you would want to transform both training data with missing values AND test data with missing values.

Give yourself time to play with different versions of the above--the essence of successful machine learning is iterative development and empirical testing!

IMPORTANT WARNING: You will be penalized for performing unprincipled preprocessing, like 1-hot encoding the CONTINUOUS variables, which would make for a hugely-parameterized and overfit model.

✅ Problem 8: Testing & Tuning

Apropos, let's now head over to the testing and tuning phase!

To run your model on the test set, execute: `pytest -k test_salary_performance`, which will run the same test but 3 times for 3 different difficulty levels, expecting you to exceed accuracies of [0.80, 0.82, 0.84] for maximum credit.

As you continue to try different preprocessing, adding or removing features, etc., re-run the above as tests of your efficacy.

To help guide your explorations as heuristics:

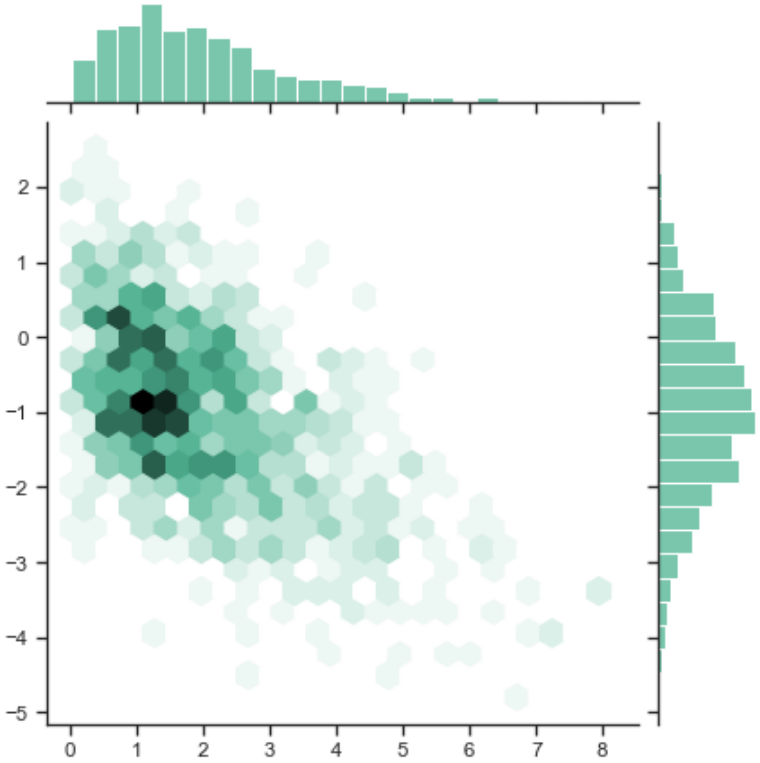
Exploratory Data Analysis and its close cousin **Data Visualization** are tools that allow you to understand the relationships between your features and class.

Visualization Tools 1: You can get some basic information about some dataset within a Pandas dataframe through the `df.info()` and `df.describe()` methods, which provide some descriptive statistics like the data types, means, and standard deviations.

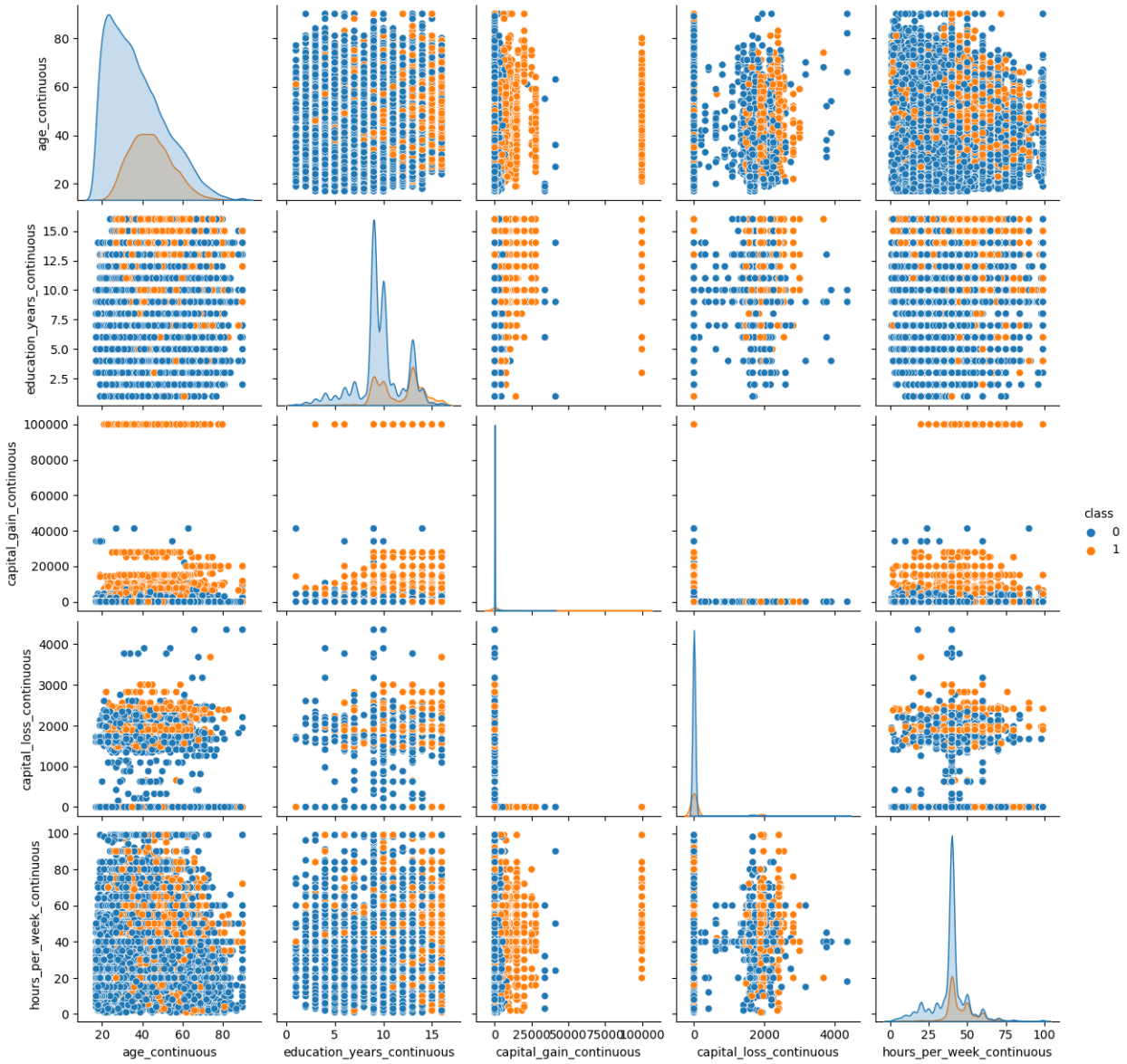
Visualization Tools 2: The `seaborn, plotly, matplotlib` libraries, among others, are great tools for data visualization.

Examples using the Seaborn library:

- The jointplot method can be used to examine the covariance between pairs of variables, and the fact that they can be customized to look like little hexes makes it look extra scientific.



- 🔗 Heatmaps can examine correlations between variables of interest.
- Here's an example of a pairplot to look at the relationships between the raw input variables and the class in our assignment dataset (generated via `seaborn.pairplot(data, hue="class").savefig("pairs.png")` where data is a dataframe of selected, *sanitized* data and the named parameter "hue" is used to color the different samples based on the class variable).



Some notes on the above:

- Sometimes these pairwise visualizations can give you insights for how to engineer features; e.g., examining the capital gains variable seems to be highly indicative of those with class Y=1 (i.e., the greater than 50k class).
- You can also gain a gist for the distributions of variables individually, e.g., that education years tends to be a bimodal distribution centered around key educational milestones like graduating with a high-school and college degree. Note also the distribution of our class variable within.
- As a rule of thumb, we generally want to avoid having features that are **correlated with one another** (i.e., LR models perform better when the features are independent from

one another), and cleanly **separate the classes** (i.e., are more highly dependent on the class). Here's where the experimentation happens!

- Moreover, because the *scales* of certain variables are different than others (e.g., age will be a number usually between 18-100 vs. capital gains which may be something huge like 1800), it is often smart to normalize / rescale continuous variables to be aligned with the others; see this section of the sklearn preprocessing for more information and examples: Dataset Standardization
- Other features sometimes require that we invest a little more sweat-equity to get meaningful predictive power...

Have fun experimenting, that's the name of the game!

Tests & Grading

Task 1: Toxicity Filter

To get full credit here: just pass the Toxicity Filter tests provided in the `classifier_tests.py`! I.e., run `pytest -k toxicity`

Task 2: Salary Predictor

- Just as in Task 1, you must pass the Salary Predictor tests provided in `classifier_tests.py` BUT also...
- ...with a catch! I've reserved 1000 test samples from the dataset on which you'll be tested! A good solution will have roughly 85% precision and recall; a *great* submission will have closer to 90. **You will receive full credit for having 85% or greater accuracy on this held-out data.**

Hint: most of your accuracy will come from intelligently selecting / engineering features from the raw data!

Submission

You will be submitting your assignments through GitHub Classroom!

What

Complete all classes that accomplishes the specification above in your `spam_filter.py` and `salary_predictor.py` given in the skeleton above.

How

To **clone** this assignment (if you need a refresher), consult the 🔗GitHub Classroom Tutorial.

To **submit** this assignment:

- Simply push your final, submission copy to the GitHub Classroom repository associated with you or your group.
- If you worked in a group, ensure that your GitHub Classroom group includes all members, and place all group members' names at the top of *all* submitted files (in appropriate commenting fashion).